

Your AI Agents Are Useless Without This: PydanticAI + FastAPI Crash Course



Launch Intelligence
10.9K subscribers

Join

Subscribe

157



Share

Ask

Download



4,679 views Apr 8, 2025 [Members first on March 6, 2025](#) #AI #APIs #Python

Join AI Dev Skool & Launch Your AI Startup Today! <https://skool.com/ai-software-developers> is the community for founders, builders, and AI innovators ready to take their projects to the next level.

If you're launching an AI startup or working on a side project, stop wasting time on endless tutorials and start focusing on what really matters. Inside AI Dev Skool, you'll:

- ✓ Get expert guidance on the best AI frameworks
- ✓ Cut through the hype and go straight to what works
- ✓ Maximize your time with curated resources and real-world insights
- ✓ Build strong connections with like-minded developers and founders

Our best members actively engage, share, and build—gaining skills while turning ideas into real businesses. If you're serious about AI development and want a shortcut to success, this is the place for you.

✈ Join now and start building smarter: <https://skool.com/ai-software-developers>

APIs are the backbone of modern AI applications, enabling seamless integration, automation, and scalability. In this tutorial, we'll dive into building AI agent APIs using FastAPI and PydanticAI—turning isolated AI models into powerful, interactive systems.

★ Topics Covered:

- ✚ FastAPI vs. Django vs. Flask – Best API framework for AI
- ✚ Creating an AI agent & exposing it via an API
- ✚ Testing endpoints with Postman
- ✚ Adding query parameters for model control
- ✚ Securing & deploying AI-powered APIs

📺 Full-Stack Agentic AI Masterclass Series:

- Part 1: Introduction
- Part 2: The AI framework landscape in 2025

- Part 3: Building effective agents - principles & practice
- Part 4: Prompt Engineering for effective agents
- Part 5: [Your AI Agents Are Useless Without This: P...](#)
- Part 6: Building UIs for your agents
- Part 7: [Stop Building Dumb Agents: Easy Database C...](#)
- Part 7.5: [AI That Writes SQL: Automate Queries with ...](#)
- Part 8: RAG
- Part 9: Integrating PydanticAI with N8N
- Part 10: Containers and deployment
- Part 11: E2E Final Project - customer support agent with PydanticAI



Full-Stack Agentic AI with PydanticAI

- The framework landscape in 2025
- Building effective agents – principles & practice
- Prompt Engineering for Better Results
- Building agent APIs – PydanticAI, FastAPI
- Building agent UI – PydanticAI, Streamlit
- Building DB agents with PydanticAI
- RAG with PydanticAI
- Integrating PydanticAI with N8N
- Containers and deployment
- Project: Customer support agent w/ PydanticAI



AI Agent APIs

Full-Stack Agentic AI Masterclass

AI SOFTWARE DEV

Skool
Channel
Newsletter

WHY an API?

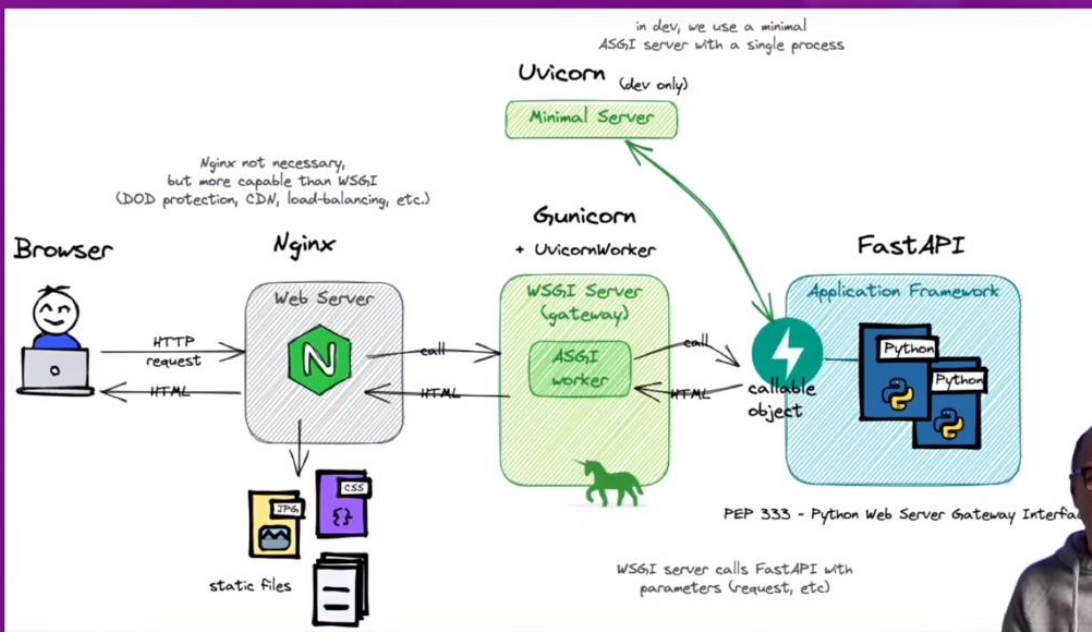
Better integration, automation, and scalability

1. **Interoperability** – APIs allow different applications, platforms, and services to work together
2. **Scalability** – Businesses can scale more efficiently by leveraging APIs to integrate new features
3. **Automation** – APIs enable automation by allowing software to interact without human intervention
4. **Innovation** – Developers can build on top of existing platforms, creating new services, AI-powered applications, and custom workflows
5. **Security** – Authentication and access controls ensure secure interactions between systems
6. **Efficiency** – Enable reuse, such as payment processing, authentication, or AI capabilities



FastAPI Primer

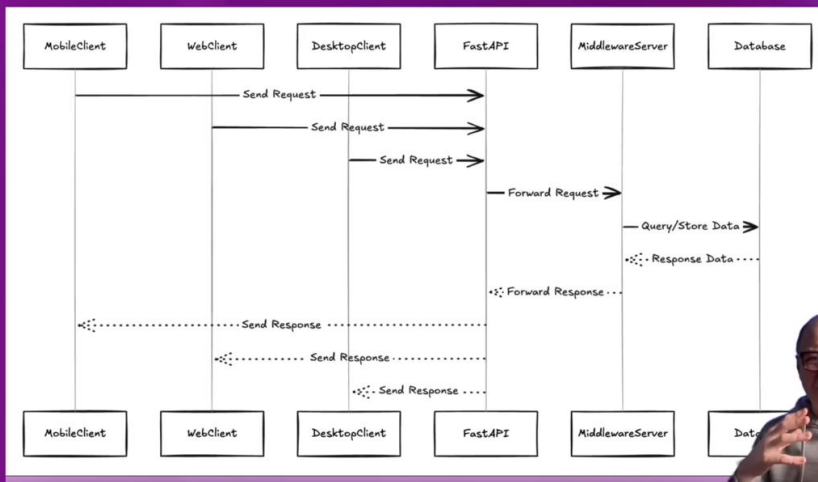
Easy API management with Python



ASGI – asynchronous server gateway interface

Interaction Sequence

Easy API management with Python



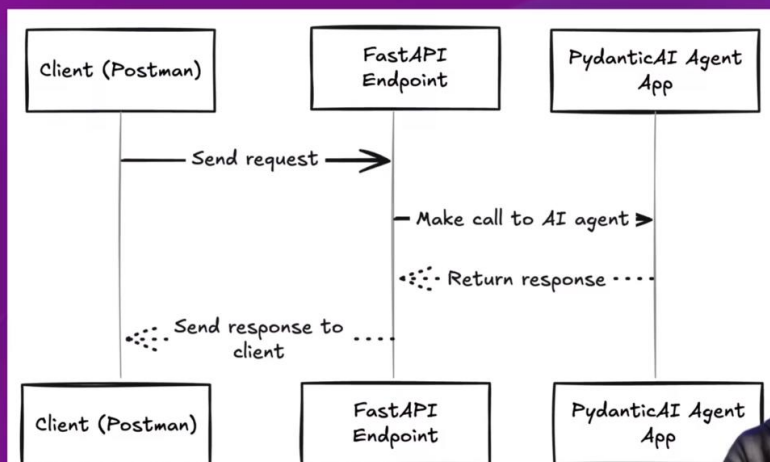
API Frameworks Compared

FastAPI vs. Django vs. Flask

	django	Flask	FastAPI
Performance speed	Normal	Faster than Django	The fastest out there
Async support	YES with restricted latency	NO needs Asyncio	YES native async support
Packages	Plenty for robust web apps	Less than Django for minimalistic apps	The least of all for building web apps
Popularity	The most popular	The second popular	Relatively new
Learning	Hard to learn	Easier to learn	The easiest to

FastAPI with PydanticAI

APIs for AI Agents



Coding Tutorial

FastAPI with PydanticAI

1. **Create an agent** – write a regular PydanticAI agent
2. **Test the agent** – basic test for the agent in the terminal
3. **Create FastAPI endpoint** – write a basic FastAPI endpoint with a single query param
4. **Test FastAPI endpoint** – test the FastAPI endpoint with Postman
5. **Add query params** – add extra query params to the endpoint – model and system prompt
6. **Test the modified FastAPI endpoint** – run Postman tests to verify changes



Create an agent

```
1 import os
2 from dotenv import load_dotenv
3 from colorama import Fore
4 from dataclasses import dataclass
5 from pydantic_ai import Agent
6 from pydantic_ai.models.groq import GroqModel
7
8 # Initialize model
9 load_dotenv()
10 GROQ_API_KEY = os.getenv("GROQ_API_KEY")
11
12 #
```

```
8 # Initialize model
9 load_dotenv()
10 GROQ_API_KEY = os.getenv("GROQ_API_KEY")
11
12 # Pydantic model for Movie
13 @dataclass
14 class Movie:
15     title: str
16     year: int
17     rating: float
18     genre: str
19     cast: list[str]
20
21 if not GROQ_API_KEY:
22     raise ValueError(
23         Fore.RED
```

```
20
21 if not GROQ_API_KEY:
22     raise ValueError(
23         Fore.RED + "Groq API key not found. Please set the GROQ_API_KEY environment variable."
24     )
25
26 def get_model(model_name:str):
27     try:
28         model = GroqModel(
29             model_name=model_name,
30             api_key=GROQ_API_KEY)
31     except Exception as e:
32         print(Fore.RED + "Error initializing model: ", e)
33         model = None
34
35     return model
36
37 # Define the main loop
38 def main_loop():
39     system_prompt = "You are a mov
```

```
36
37 # Define the main loop
38 def main_loop():
39     system_prompt = "You are a movie critic and expert."
40     agent = Agent(model=get_model("llama-3.3-70b-versatile"), result_type=Movie, system_prompt=system_prompt)
41     while True:
42         user_input = input(">> Enter a movie query (q, quit, exit to exit): ")
43         if user_input.lower() in ["quit", "exit", "q"]:
44             print("Goodbye!")
45             break
46         result = agent.run_sync(user_input)
47         print(Fore.YELLOW, result.data)
48
49 if __name__ == "__main__":
50     main_loop()
```

```
36
37 # Define the main loop
38 def main_loop():
39     system_prompt = "You are a movie critic and expert."
40     agent = Agent(model=get_model("llama-3.3-70b-versatile"), result_type=Movie, system_prompt=system_prompt)
41     while True:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

zsh - 5-agent-api

(myenv) 9:37:12 ~/dev/tuts/full-stack-ai-masterclass/5-agent-api \$ clear && poetry run python agent.py

```
36
37 # Define the main loop
38 def main_loop():
39     system_prompt = "You are a movie critic and expert."
40     agent = Agent(model=get_model("llama-3.3-70b-versatile"), result_type=Movie, system_prompt=system_prompt)
41     while True:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

python3.12 - 5-agent-api

>> Enter a movie query (q, quit, exit to exit): Ocean's Eleven
Movie(title="Ocean's Eleven", year=2001, rating=7.7, genre='Heist', cast=['George Clooney', 'Brad Pitt', 'Julia Roberts'])
>> Enter a movie query (q, quit, exit to exit): Mission Impossible Ghost Protocol
Movie(title='Mission Impossible Ghost Protocol', year=2011, rating=7.9, genre='Action', cast=['Tom Cruise', 'Jeremy Renner', 'Simon Pegg', 'Paula Patton'])
>> Enter a movie query (q, quit, exit to exit):

This works as expected!

Create an API

```
1 from fastapi import FastAPI
2 from agent import getMovieDetails
3
4 app = FastAPI()
5
6 # FastAPI endpoint
7 @app.get("/getMovieDetails/")
8 async def get_movie_details_endpoint(q: str | None = None):
9     return await getMovieDetails(q)
```

```
38 def main_loop():
46     result = agent.run_sync(user_input)
47     print(Fore.YELLOW, result.data)
48
49 if __name__ == "__main__":
50     main_loop()
51
52 # FastAPI endpoint method
53 async def getMovieDetails(q: str) -> Movie:
54     model_name = "llama-3.3-70b-versatile"
55     system_prompt = "You are a movie critic and expert."
56     agent = Agent(model=get_model(model_name), result_type=Movie, system_prompt=system_prompt)
57     result = await agent.run(q)
58     return result.data
```

```
1 from fastapi import FastAPI
2 from agent import getMovieDetails
3
4 app = FastAPI()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

zsh - 5-agent-api

(myenv) 9:49:43 ~/dev/tuts/full-stack-ai-masterclass/5-agent-api \$ clear && poetry run fastapi dev api.py

We will be using poetry and a **.toml** file to run our **FastAPI** application

```
1 from fastapi import FastAPI
2 from agent import getMovieDetails
3
4 app = FastAPI()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

python3.12 - 5-agent-api

FastAPI Starting development server 🚀

Searching for package file structure from directories with `__init__.py` files
Importing from `/Users/robsmith/dev/tuts/full-stack-ai-masterclass/5-agent-api`

module `api.py`

code Importing the FastAPI app object from the module with the following code: `from api import app`

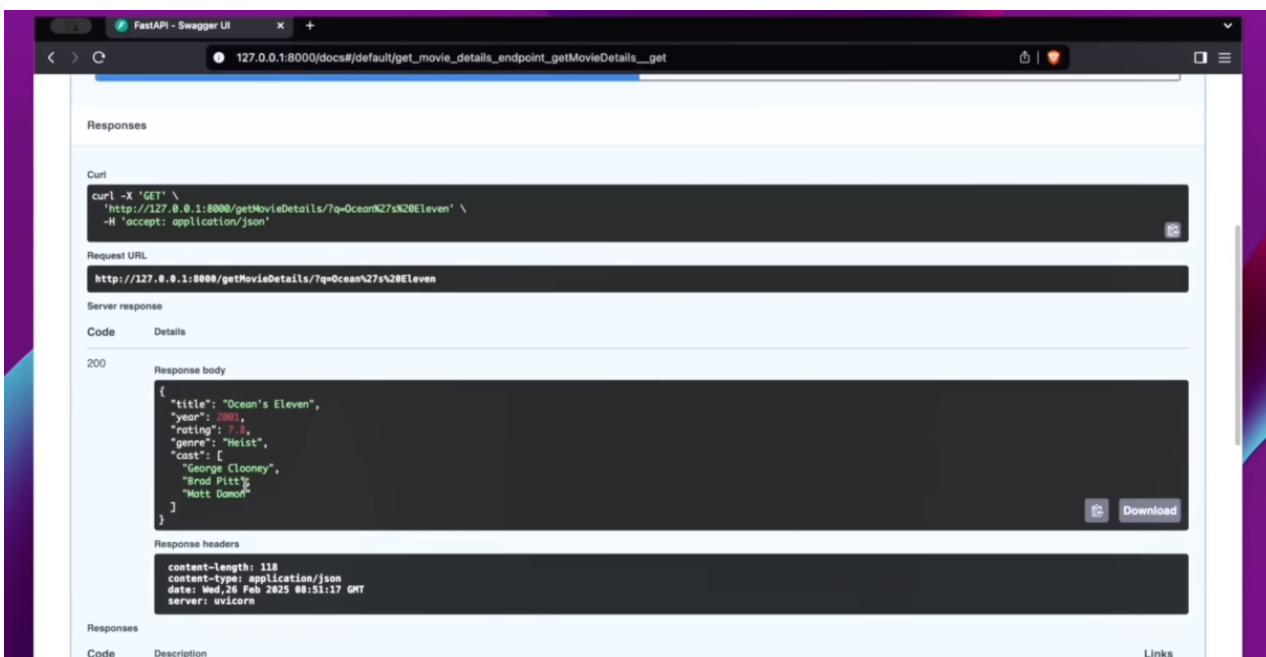
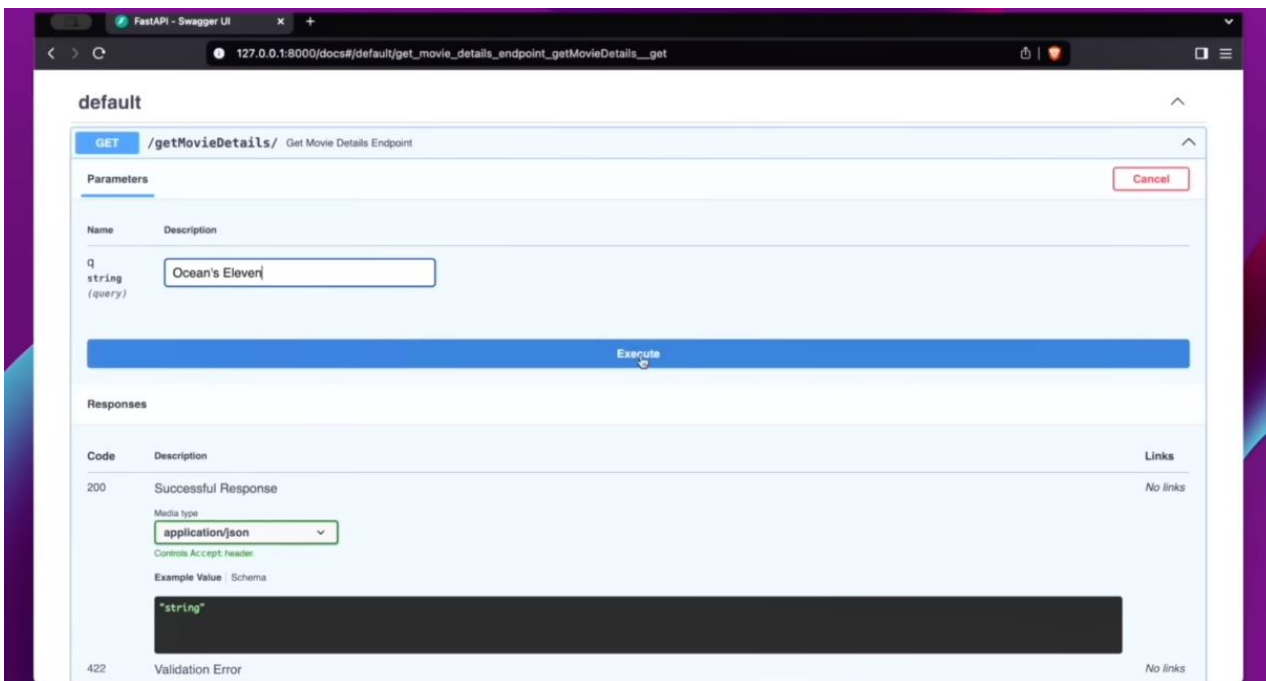
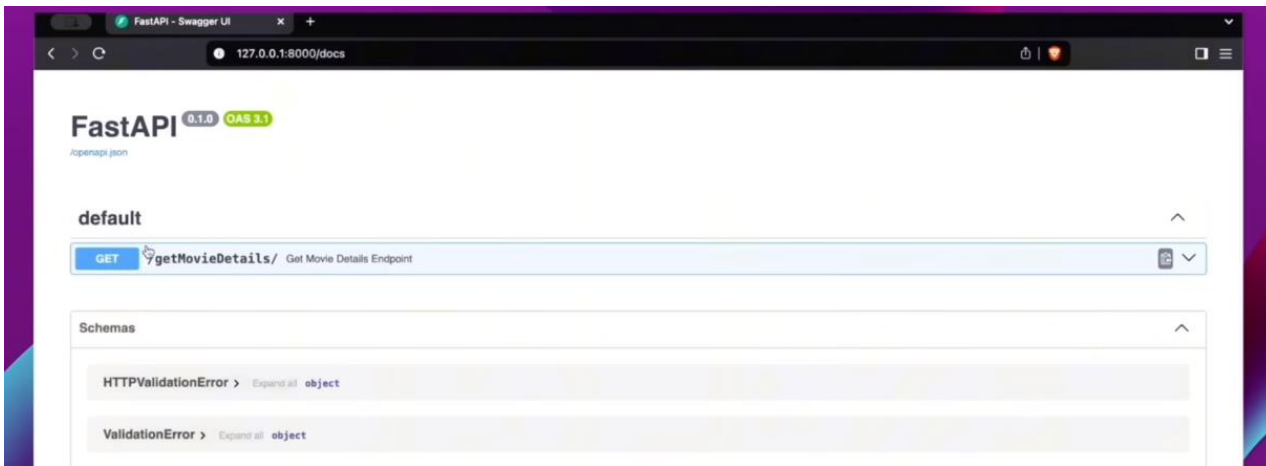
app Using import string: `api:app`

server Server started at <http://127.0.0.1:8000>
Documentation at <http://127.0.0.1:8000/docs>

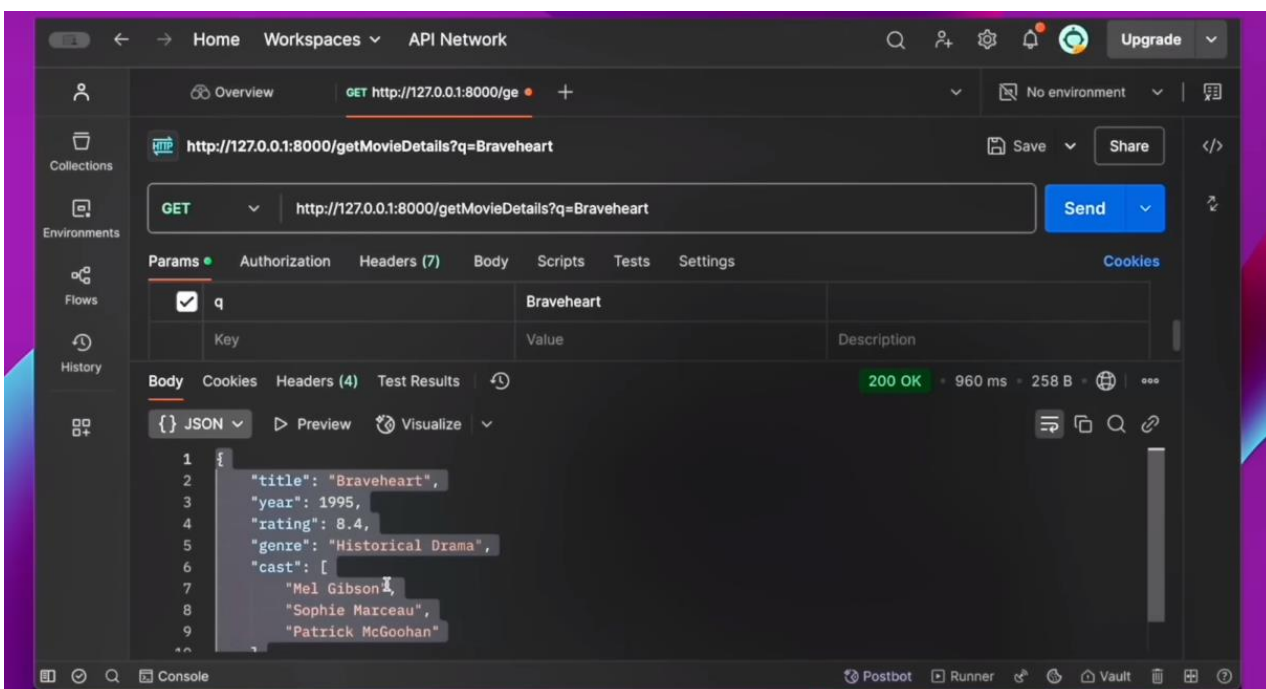
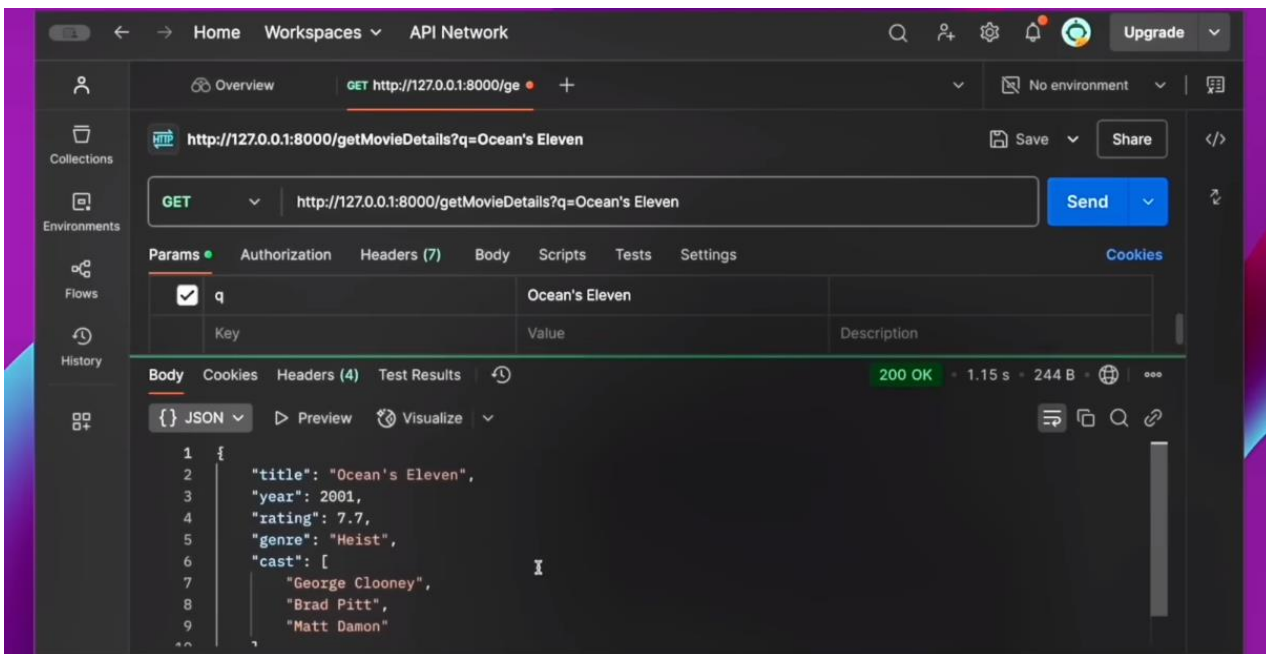
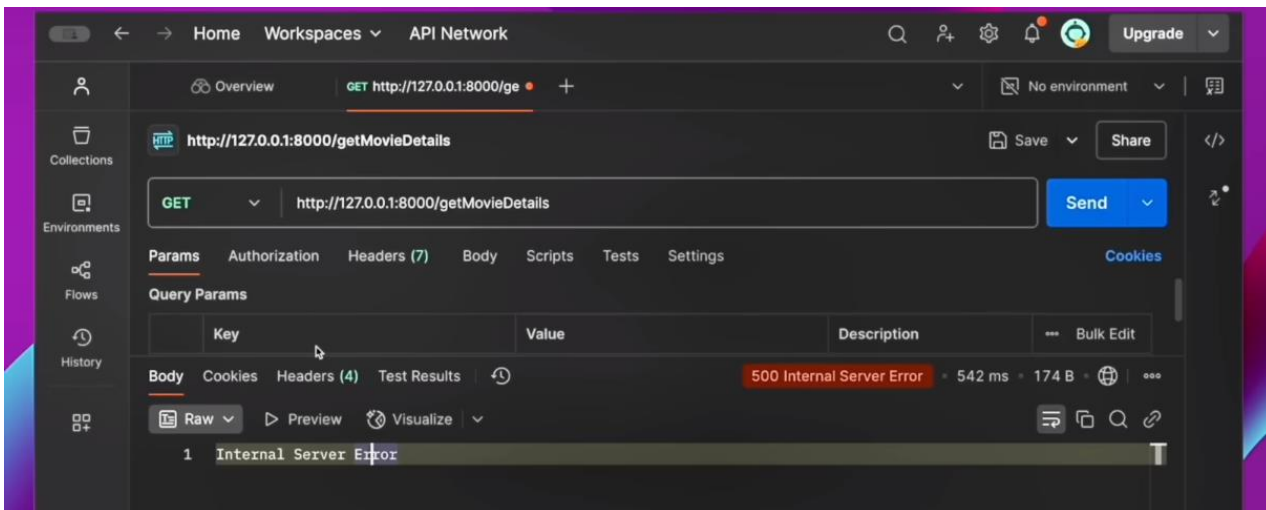
tip Running in development mode, for production use: `fastapi run`

Logs:

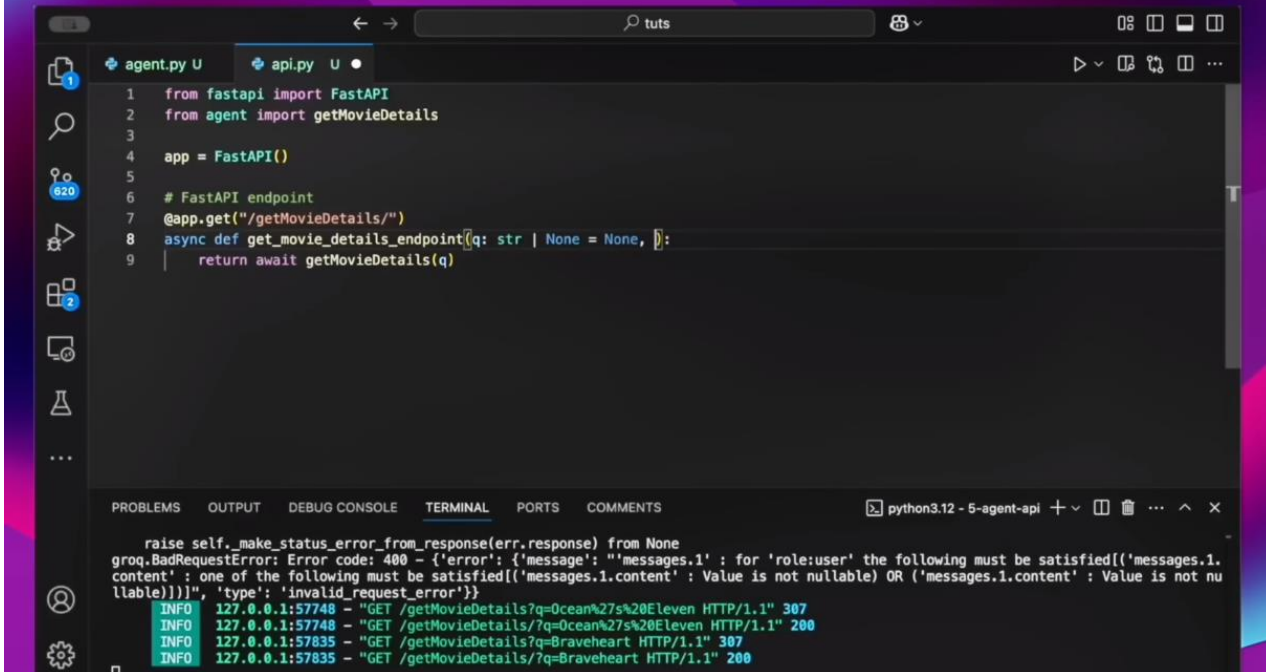
INFO Will watch for changes in these directories: `['/Users/robsmith/dev/tuts/full-stack-ai-masterclass/5-agent-api']`
INFO Uvicorn running on <http://127.0.0.1:8000> (Press CTRL+C to quit)
INFO Started reloader process `[60587]` using WatchFiles
INFO Started server process `[60625]`
INFO Waiting for application startup.
INFO Application startup complete.



Our FastAPI endpoint is working successfully, we also have a curl command to use for automation



Add model to the API



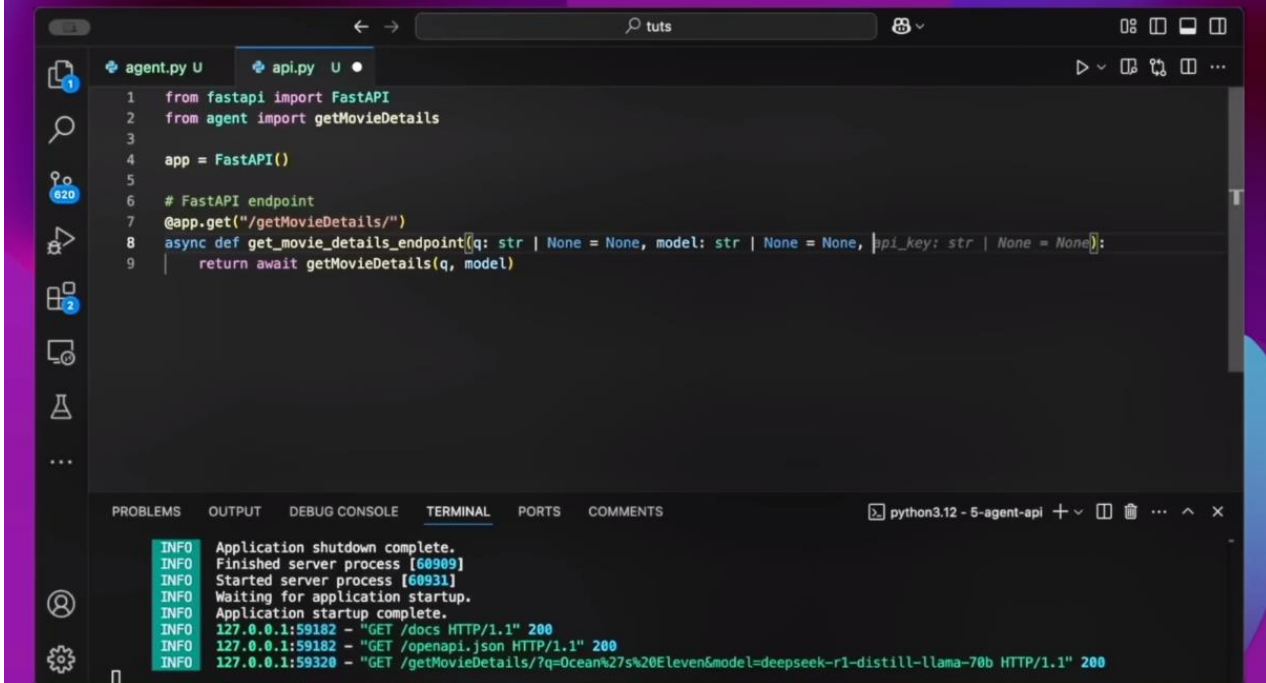
```
1 from fastapi import FastAPI
2 from agent import getMovieDetails
3
4 app = FastAPI()
5
6 # FastAPI endpoint
7 @app.get("/getMovieDetails/")
8 async def get_movie_details_endpoint(q: str | None = None, ):
9     return await getMovieDetails(q)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS python3.12 - 5-agent-api

```
raise self._make_status_error_from_response(err.response) from None
groq.BadRequestError: Error code: 400 - {'error': {'message': "'messages.1' : for 'role:user' the following must be satisfied[('messages.1.content' : one of the following must be satisfied[('messages.1.content' : Value is not nullable) OR ('messages.1.content' : Value is not nullable)]]", 'type': 'invalid_request_error'}}
INFO 127.0.0.1:57748 - "GET /getMovieDetails?q=Ocean%27s%20Eleven HTTP/1.1" 307
INFO 127.0.0.1:57748 - "GET /getMovieDetails/?q=Ocean%27s%20Eleven HTTP/1.1" 200
INFO 127.0.0.1:57835 - "GET /getMovieDetails?q=Braveheart HTTP/1.1" 307
INFO 127.0.0.1:57835 - "GET /getMovieDetails/?q=Braveheart HTTP/1.1" 200
```

We can also use FastAPI to change the agent model we want to use as an optional parameter as below

Add prompt to the API

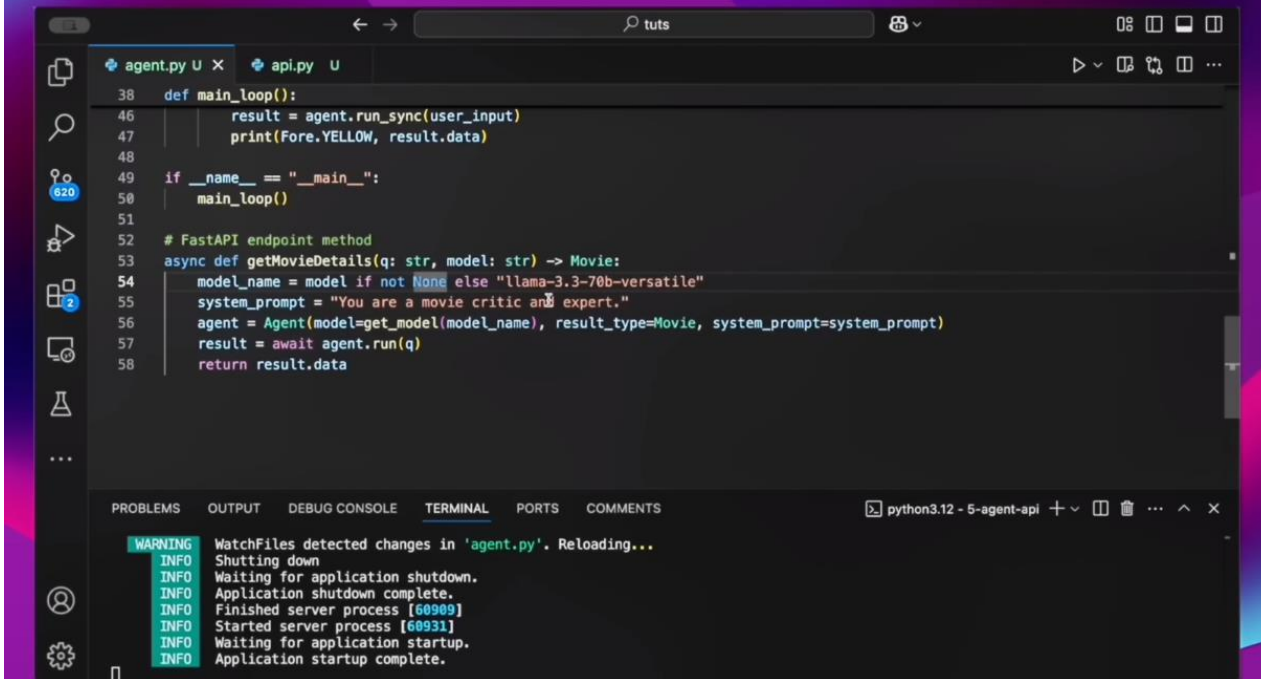


```
1 from fastapi import FastAPI
2 from agent import getMovieDetails
3
4 app = FastAPI()
5
6 # FastAPI endpoint
7 @app.get("/getMovieDetails/")
8 async def get_movie_details_endpoint(q: str | None = None, model: str | None = None, api_key: str | None = None):
9     return await getMovieDetails(q, model)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS python3.12 - 5-agent-api

```
INFO Application shutdown complete.
INFO Finished server process [60909]
INFO Started server process [60931]
INFO Waiting for application startup.
INFO Application startup complete.
INFO 127.0.0.1:59182 - "GET /docs HTTP/1.1" 200
INFO 127.0.0.1:59182 - "GET /openapi.json HTTP/1.1" 200
INFO 127.0.0.1:59320 - "GET /getMovieDetails/?q=Ocean%27s%20Eleven&model=deepseek-r1-distill-llama-70b HTTP/1.1" 200
```

Add model to the API

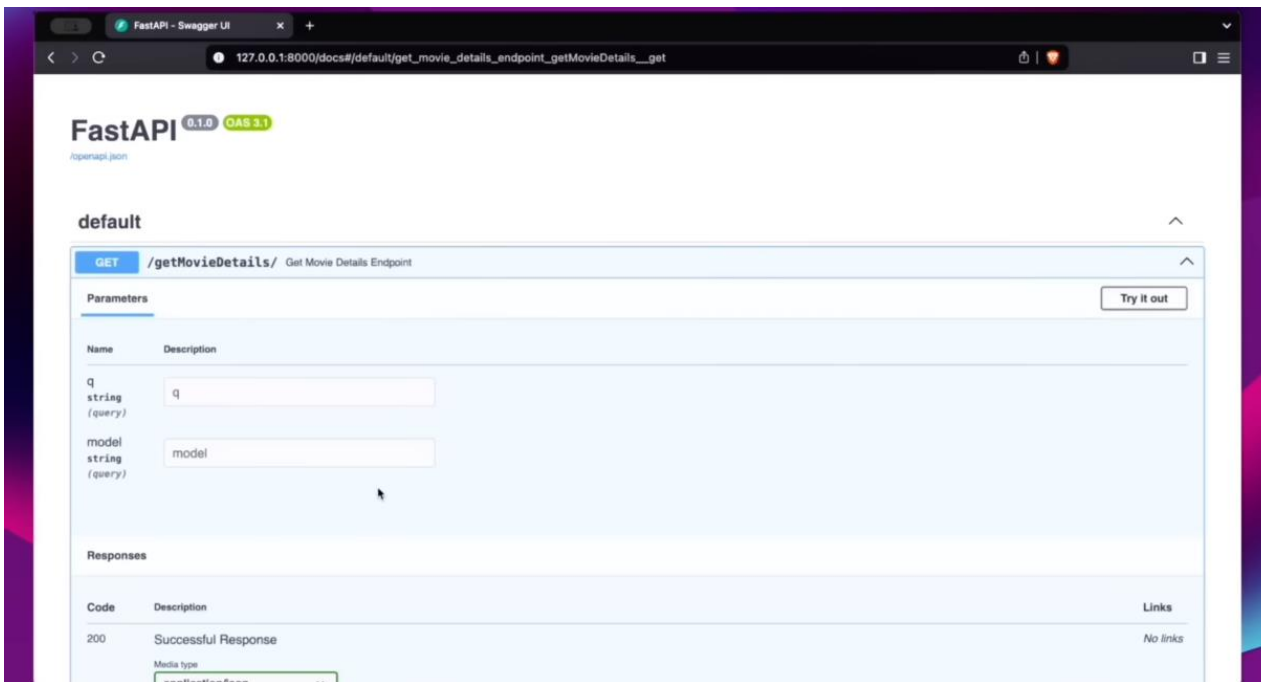


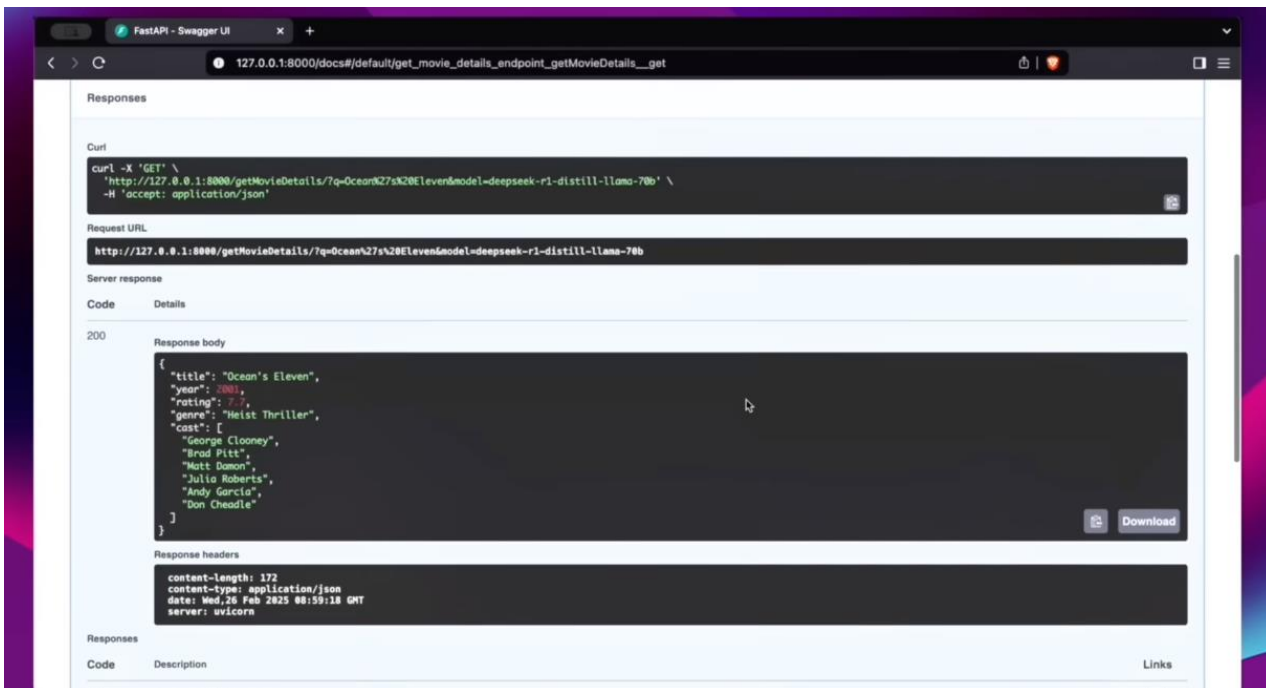
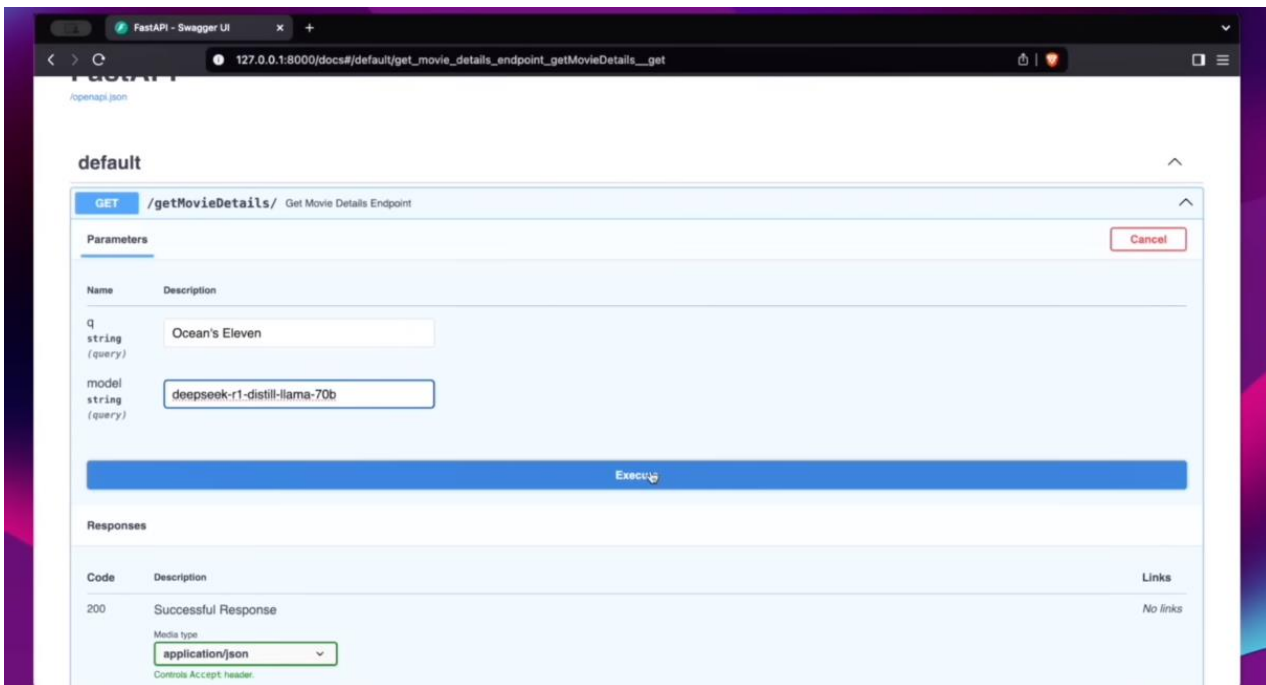
The screenshot shows a code editor with two files: `agent.py` and `api.py`. The `api.py` file contains the following code:

```
38 def main_loop():
46     result = agent.run_sync(user_input)
47     print(Fore.YELLOW, result.data)
48
49 if __name__ == "__main__":
50     main_loop()
51
52 # FastAPI endpoint method
53 async def getMovieDetails(q: str, model: str) -> Movie:
54     model_name = model if not None else "llama-3.3-70b-versatile"
55     system_prompt = "You are a movie critic and expert."
56     agent = Agent(model=get_model(model_name), result_type=Movie, system_prompt=system_prompt)
57     result = await agent.run(q)
58     return result.data
```

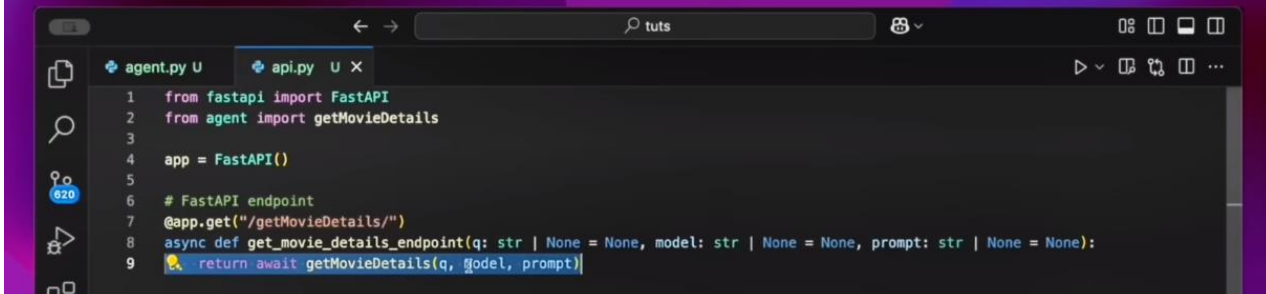
The terminal output shows the following messages:

```
WARNING: WatchFiles detected changes in 'agent.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [60909]
INFO: Started server process [60931]
INFO: Waiting for application startup.
INFO: Application startup complete.
```





Add prompt to the API



We can also parameterize the system prompt as above too

```
agent.py U  apl.py U
38 def main_loop():
46     result = agent.run_sync(user_input)
47     print(Fore.YELLOW, result.data)
48
49 if __name__ == "__main__":
50     main_loop()
51
52 # FastAPI endpoint method
53 async def getMovieDetails(q: str, model: str, prompt: str) -> Movie:
54     model_name = model if not None else "llama-3.3-70b-versatile"
55     system_prompt = prompt if not None else "You are a movie critic and expert."
56     agent = Agent(model=get_model(model_name), result_type=Movie, system_prompt=system_prompt)
57     result = await agent.run(q)
58     return result.data
```

FastAPI - Swagger UI

127.0.0.1:8000/docs#/default/get_movie_details_endpoint_getMovieDetails__get

FastAPI 0.1.0 OAS 3.1

default

GET /getMovieDetails/ Get Movie Details Endpoint

Try it out

Parameters

Name	Description
q	q
model	model
prompt	prompt

Responses

Code	Description
------	-------------

Links

FastAPI - Swagger UI

127.0.0.1:8000/docs#/default/get_movie_details_endpoint_getMovieDetails__get

FastAPI 0.1.0 OAS 3.1

default

GET /getMovieDetails/ Get Movie Details Endpoint

Cancel

Parameters

Name	Description
q	Ocean's Eleven
model	deepseek-r1-distill-llama-70b
prompt	Always add Marty The Braveheart to the cast

Execute

Responses

FastAPI - Swagger UI

127.0.0.1:8000/docs#/default/get_movie_details_endpoint_getMovieDetails__get

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/getMovieDetails/?q=Ocean's Eleven&model=deepseek-r1-distill-llama-70b&prompt=Always%20add%20Marty%20The%20Braveheart%20to%20the%20cast' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/getMovieDetails/?q=Ocean's Eleven&model=deepseek-r1-distill-llama-70b&prompt=Always%20add%20Marty%20The%20Braveheart%20to%20the%20cast
```

Server response

Code	Details
200	Response body <pre>{ "title": "Ocean's Eleven", "year": 2001, "rating": 4.5, "genre": "Heist/Crime", "cast": ["George Clooney", "Brad Pitt", "Julie Roberts", "Marty The Braveheart"] }</pre> Response headers <pre>content-length: 158 content-type: application/json date: Wed, 26 Feb 2025 09:01:43 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

FastAPI 0.110.0 (0.110.0)

default

GET /getMovieDetails/ Get Movie Details Endpoint

Parameters

Name	Description
q string (query)	Ocean's Eleven
model string (query)	deepseek-r1-distill-llama-70b
prompt string (query)	Limit the cast to 1 member

Execute Clear

Responses

Curl

```
curl -X 'GET' \
```

FastAPI - Swagger UI

127.0.0.1:8000/docs#/default/get_movie_details_endpoint_getMovieDetails__get

Curl

```
curl -X 'GET' \
  "http://127.0.0.1:8000/getMovieDetails/?q=Ocean's%20Eleven&model=deepseek-r1-distill-llama-70b&prompt=Limit%20the%20cast%20to%201%20member" \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/getMovieDetails/?q=Ocean's%20Eleven&model=deepseek-r1-distill-llama-70b&prompt=Limit%20the%20cast%20to%201%20member

Server response

Code	Details
200	Response body <pre>{ "title": "Ocean's Eleven", "year": 2001, "rating": 8.2, "genre": "Heist", "cast": ["George Clooney"] }</pre> Response headers <pre>content-length: 93 content-type: application/json date: Mon, 26 Feb 2025 09:02:03 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header.

FastAPI - Swagger UI

127.0.0.1:8000/docs#/default/get_movie_details_endpoint_getMovieDetails__get

FastAPI 0.109.0 OAS 3.1

/openapi.json

default

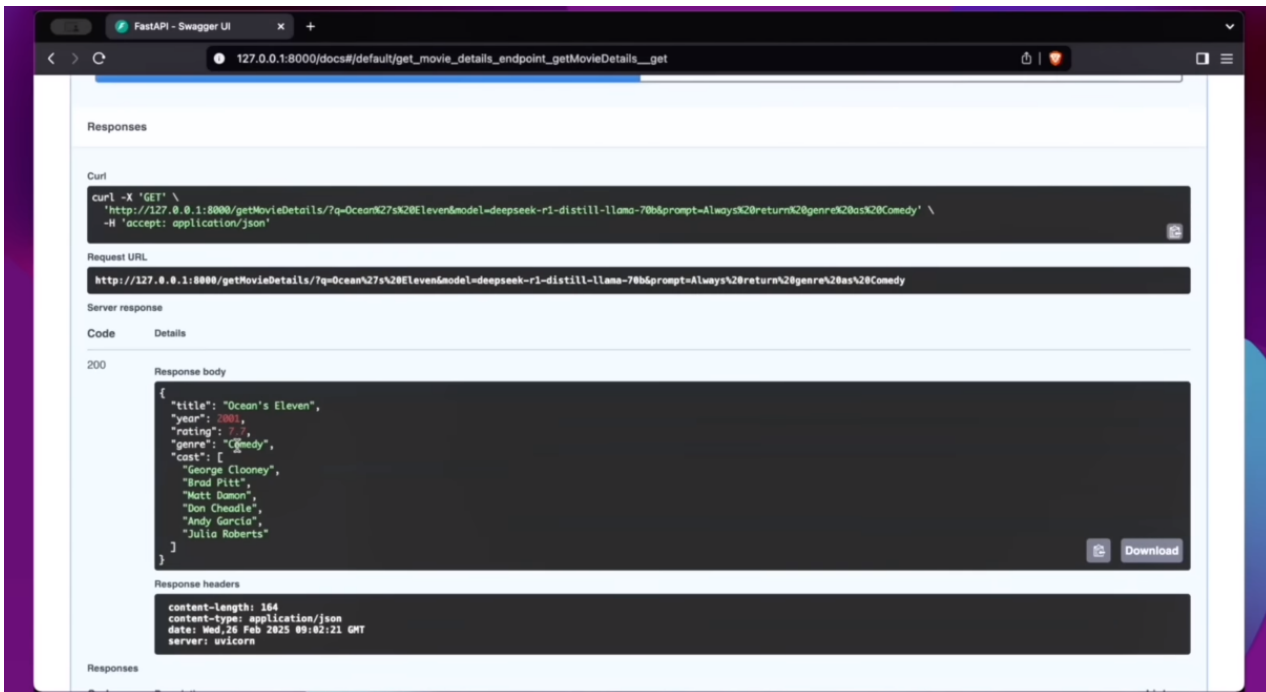
GET /getMovieDetails/ Get Movie Details Endpoint

Parameters

Name	Description
q string (query)	Ocean's Eleven
model string (query)	deepseek-r1-distill-llama-70b
prompt string (query)	Always return genre as Comedy

Execute Clear

Responses



We can now start integrating various agents to our workflow using this same FastAPI implementation

Summary

FastAPI with PydanticAI

APIs aren't just a convenience — they're the backbone of modern software. They enable interoperability, scalability, automation, security, and innovation, allowing developers to build powerful, connected systems.

Whether you're integrating AI, streamlining workflows, or scaling your applications, **APIs unlock new possibilities**. The right API strategy can mean the difference between isolated agents and truly intelligent, automated connected AI systems.

